



FIAP · Pós-Tech

Arquitetura e Desenvolvimento em Java

TECH CHALLENGE — FASE 3

Ecossistema de Microserviços Hospitalares

Documentação técnica da AgendamentoAPI e da HistoricoAPI. Reúne o agendamento de consultas com autenticação JWT e mensageria, e a consulta do histórico do paciente por GraphQL, sobre um banco PostgreSQL compartilhado.

DISCIPLINA

Arquitetura e Desenvolvimento em Java

TURMA E GRUPO

Turma 12ADJT · Grupo 46

INTEGRANTES

Gustavo Chauar Correa (RM 370978)
Eduardo Mendes Germano (RM 371760)
Davi Faustino Souza Lima (RM 373019)
Caroline Da Silva Brunheti (RM 372744)
Roberto Domingos Locatelli (RM 370928)

REPOSITÓRIOS

github.com/JavaPosTech/AgendamentoAPI
github.com/JavaPosTech/HistoricoAPI

ENTREGA E VERSÃO

Setembro de 2026 · versão 1.0.0

Sumário

1. Introdução	3
Descrição do problema · Objetivo · Os dois microsserviços	3
2. Arquitetura do Sistema	4
Camadas · Tecnologias · Decisões · Topologia de contêineres	4
3. AgendamentoAPI: Serviço de Agendamento	7
Domínios · Estrutura de pastas · Segurança · Endpoints · Regras de negócio · Mensageria · Exemplos	7
4. HistoricoAPI: Serviço de Histórico	12
Estrutura de pastas · GraphQL · Endpoints · Erros previstos · Exemplos	12
5. Configuração e Execução	15
Docker Compose · Ordem de subida · Passo a passo · URLs e portas	15
6. Qualidade do Código	18
Boas práticas · SOLID e DRY · Testes e cobertura · Integração contínua	18
7. Repositórios do Código	19

1. Introdução

Descrição do problema

Em um ambiente hospitalar é essencial contar com sistemas que garantam o **agendamento eficaz de consultas**, o **gerenciamento do histórico do paciente** e a **comunicação assíncrona** entre os serviços que reagem a esses eventos. O sistema deve ser acessível a diferentes tipos de usuários (administradores, médicos, enfermeiros, recepcionistas e pacientes), com acesso controlado e funcionalidades específicas para cada perfil.

Para atender a esse cenário, a solução da fase é decomposta em **microsserviços**. O atendimento gera informação em pontos distintos do fluxo: o cadastro das pessoas, o agendamento das consultas e o registro do histórico clínico. Cada recorte é responsabilidade de um serviço próprio, com o seu ciclo de build, imagem e implantação independentes, porém todos apoiados em uma **fonte única de verdade**: um banco de dados PostgreSQL compartilhado.

Objetivo do projeto

Desenvolver um backend **modular e seguro**, aplicando boas práticas de autenticação, autorização, comunicação entre serviços e containerização, de modo que cada serviço possa escalar e ser substituído de forma independente. Concretamente, a fase entrega dois microsserviços complementares:

- **AgendamentoAPI**: API REST que cadastra pessoas (médicos, enfermeiros, recepcionistas e pacientes), autentica usuários por JWT, agenda e remarca consultas, registra o histórico clínico e publica eventos de agendamento em uma fila RabbitMQ. É a **dona do schema** do banco compartilhado.
- **HistoricoAPI**: API GraphQL, somente leitura, que consolida em uma única consulta os dados cadastrais do paciente, o histórico clínico e as consultas agendadas, lendo o mesmo banco sem duplicar a responsabilidade de evoluí-lo.

Divisão de responsabilidades. A AgendamentoAPI concentra toda a escrita e é a única a possuir migrations Flyway, e cria as tabelas e a carga inicial. A HistoricoAPI apenas mapeia o schema já existente e responde por consultas. Essa separação evita que dois serviços disputem a evolução do banco.

Os dois microsserviços em um olhar

Serviço	Responsabilidade	Interface	Porta (host)
AgendamentoAPI	Cadastros, autenticação JWT, agendamento de consultas, histórico e publicação de eventos. Dona das migrations.	REST + OpenAPI	9027
HistoricoAPI	Consulta consolidada do histórico do paciente. Somente leitura.	GraphQL	9028

2. Arquitetura do Sistema

Descrição da arquitetura

Os dois serviços seguem a **arquitetura em camadas** do ecossistema Spring, com direção de dependência única: cada camada conhece apenas a imediatamente abaixo. O código é organizado **por camada e, dentro de cada camada, por domínio**, então a estrutura de pacotes revela ao mesmo tempo a responsabilidade técnica e o assunto de negócio. Na AgendamentoAPI, criar um novo domínio significa replicar a fatia vertical completa (entidade → repositório → mapper → request/DTO → serviço → controller → migration → testes).

Camadas da aplicação

Camada	Responsabilidade
Controller	Fronteira da API. Na AgendamentoAPI, endpoints REST que delegam ao serviço. Na HistoricoAPI, o resolver GraphQL. Sem regra de negócio.
Service	Regra de negócio, orquestração, validações, transações (@Transactional) e logging.
Repository	Acesso a dados declarativo, por interfaces do Spring Data JPA.
Model	Entidades JPA, DTOs de saída (records imutáveis), records de request e mappers MapStruct.
Exceptions / Config	Exceções de negócio, tratamento centralizado de erros e configuração (segurança, banco, Swagger, RabbitMQ).

Tecnologias utilizadas

Item	Tecnologia	Versão
Linguagem	Java	21
Framework	Spring Boot	4.0.5
Build	Gradle (Kotlin DSL)	9.7
Banco de dados	PostgreSQL	18
Migrações	Flyway (somente na AgendamentoAPI)	—
Interface (AgendamentoAPI)	Spring MVC (REST) + SpringDoc OpenAPI	3.0.2
Interface (HistoricoAPI)	Spring for GraphQL + SpringDoc OpenAPI	3.1.0
Segurança	Spring Security + JWT, na AgendamentoAPI	0.12.6
Mensageria	Spring AMQP + RabbitMQ, na AgendamentoAPI	—
Mapeamento	MapStruct, na AgendamentoAPI	1.6.3

Item	Tecnologia	Versão
Persistência	Spring Data JPA / Hibernate	—
Logging	Log4j2 via SLF4J (Logback excluído)	—
Testes e cobertura	JUnit 5, JaCoCo	—
Containerização	Docker / Docker Compose	—

Decisões de arquitetura

- **Banco compartilhado com dono único do schema.** As migrations Flyway pertencem à AgendamentoAPI, que cria as tabelas e a carga inicial. A HistoricoAPI não possui `ddl-auto` nem Flyway em produção, apenas mapeando o schema existente. Assim há uma fonte única de verdade e nenhum serviço disputa a evolução do banco.
- **REST para escrita, GraphQL para leitura profunda.** O agendamento é um domínio de escrita com regras e autorização por perfil, natural em REST. O histórico é um agregado profundo (paciente + histórico + consultas). Com GraphQL o cliente declara exatamente os campos desejados, em uma requisição, eliminando *over-fetching* e chamadas encadeadas.
- **Comunicação assíncrona por eventos.** Ao criar ou remarcar uma consulta, a AgendamentoAPI publica um evento em uma exchange RabbitMQ, desacoplando quem agenda de quem reage (por exemplo, um serviço de notificação por e-mail).
- **Segurança stateless com JWT.** A sessão é `STATELESS`, e o token carrega o perfil do usuário, e a autorização é declarada por método HTTP no `SecurityConfig`.
- **Carregamento explícito de relacionamentos.** Com `open-in-view: false` não há sessão JPA aberta na camada web, então as associações necessárias são carregadas dentro da transação, evitando N+1 e `LazyInitializationException`.
- **Containerização reprodutível.** Build multi-stage no Dockerfile (`gradle:jdk21-alpine` compila, `eclipse-temurin:21-jre-alpine` executa) e orquestração por Docker Compose sobre uma rede compartilhada.

Topologia de execução em contêineres

Os serviços da fase compartilham um único PostgreSQL e, no caso da AgendamentoAPI, um RabbitMQ. A infraestrutura sobe de forma independente e cria a rede **shared-net**, e cada serviço entra nela como rede externa.



Figura 1. Contêineres e rede compartilhada da Fase 3

Dentro da rede, o banco é sempre alcançado pelo hostname **postgres** na porta interna **5432** e o broker por **rabbitmq:5672**. As portas **8745**, **5672** e **15672** são apenas exposição no host. Como todas as APIs escutam em **9027** dentro do próprio contêiner, elas se diferenciam pela porta publicada: a AgendamentoAPI em **9027** e a HistoricoAPI em **9028**.

Ordem de inicialização. A infraestrutura cria a rede **shared-net**, e o Compose de cada API a consome como rede externa. Por isso a infraestrutura precisa subir primeiro, e a AgendamentoAPI deve rodar antes das demais contra um banco novo, para que as migrations criem o schema que os outros serviços consomem.

3. AgendamentoAPI: Serviço de Agendamento

API REST responsável pela escrita do domínio: cadastros, autenticação, agendamento de consultas e histórico clínico. Todas as rotas são relativas ao context path `/AgendamentoAPI`, que diferencia maiúsculas de minúsculas.

Domínios

Cada domínio possui a fatia vertical completa (entidade, repositório, mapper, request/DTO, serviço, controller e testes):

- **usuario / auth:** credenciais e login, com a senha gravada em BCrypt e nunca exposta.
- **medico, enfermeiro, recepcionista:** profissionais, com criação, edição e exclusão restritas ao administrador.
- **paciente:** cadastrado pela recepcionista e excluído pelo administrador.
- **agendamento:** marcação, remarcação e cancelamento de consultas, com regras de agenda.
- **historicopaciente:** registro clínico do paciente, consumido pela HistoricoAPI.
- **tipousuario, situacaocadastro:** tabelas de domínio espelhadas por enums de IDs fixos.

Estrutura de pastas

```
src/main/java/br/com/fiap/agendamentoapi/  
├─ config/           # SecurityConfig, DataBaseConfig, SwaggerConfig, RabbitMQConfig  
├─ controller/       # REST por domínio (agendamento, medico, paciente, ...)  
├─ service/          # regra de negócio, @Transactional, logging  
├─ repository/       # interfaces JpaRepository  
├─ model/  
│   ├── entity/      # entidades JPA (Lombok)  
│   ├── dto/         # records de saída  
│   ├── request/     # records de entrada (Salvar.../Atualizar...)  
│   ├── mapper/      # interfaces MapStruct  
│   ├── response/    # PageResponse, MensagemSucessoResponse, TokenResponse  
│   └─ rabbitmq/     # eventos publicados (AgendamentoCriado/AtualizadoEvent)  
├─ exceptions/       # exceções + GlobalExceptionHandler  
└─ enums/            # TipoUsuario, SituacaoCadastro  
src/main/resources/db/migration/ # V1.0__CreateTables.sql, V1.1__Inserts.sql
```

Segurança e perfis de acesso

A autenticação é por **JWT**. São públicos apenas `POST /v1/auth/login`, a documentação OpenAPI e a Swagger UI. Todo o resto exige token. **Não há autocadastro:** criar qualquer cadastro exige um usuário autenticado. O perfil vem do `tipoUsuario`, prefixado com `ROLE_`, e é comparado por `hasRole( ... )`. Um token de usuário excluído é recusado, pois a exclusão desativa também as credenciais.

Recurso	Criar / Editar	Excluir	Listar / Consultar
Médico, enfermeiro, recepcionista	ADMINISTRADOR	ADMINISTRADOR	Todos, exceto PACIENTE
Paciente	Criar: RECEPTIONISTA · Editar: todos exceto PACIENTE	ADMINISTRADOR	Todos, exceto PACIENTE
Histórico do paciente	MEDICO ou RECEPTIONISTA	ADMINISTRADOR	MEDICO, RECEPTIONISTA ou ENFERMEIRO
Agendamento (consulta)	Marcar: MEDICO, RECEPTIONISTA ou ENFERMEIRO · Remarcar: RECEPTIONISTA	RECEPTIONISTA	MEDICO, RECEPTIONISTA, ENFERMEIRO ou PACIENTE

O paciente vê só as próprias consultas. O `GET /v1/agendamento` é o único endpoint acessível ao perfil PACIENTE. Quando quem chama é um paciente, a resposta traz apenas as consultas dele. Os demais papéis veem todas. O administrador, por decisão de negócio, não acessa consultas nem histórico (exceto para excluir histórico).

Endpoints da API

Método	Rota	Autorização
POST	<code>/v1/auth/login</code>	Pública
GET	<code>/v1/medico</code>	Exceto PACIENTE
POST	<code>/v1/medico</code>	ADMINISTRADOR
PATCH	<code>/v1/medico/{id}</code>	ADMINISTRADOR
DELETE	<code>/v1/medico/{id}</code>	ADMINISTRADOR
GET	<code>/v1/enfermeiro</code>	Exceto PACIENTE
POST	<code>/v1/enfermeiro</code>	ADMINISTRADOR
PATCH	<code>/v1/enfermeiro/{id}</code>	ADMINISTRADOR
DELETE	<code>/v1/enfermeiro/{id}</code>	ADMINISTRADOR
GET	<code>/v1/recepcionista</code>	Exceto PACIENTE
POST	<code>/v1/recepcionista</code>	ADMINISTRADOR
PATCH	<code>/v1/recepcionista/{id}</code>	ADMINISTRADOR
DELETE	<code>/v1/recepcionista/{id}</code>	ADMINISTRADOR
GET	<code>/v1/paciente</code>	Exceto PACIENTE

Método	Rota	Autorização
POST	/v1/paciente	RECEPCIONISTA
PATCH	/v1/paciente/{id}	Exceto PACIENTE
DELETE	/v1/paciente/{id}	ADMINISTRADOR
GET	/v1/historico-paciente	MEDICO, RECEPCIONISTA, ENFERMEIRO
POST	/v1/historico-paciente	MEDICO ou RECEPCIONISTA
PATCH	/v1/historico-paciente/{id}	MEDICO ou RECEPCIONISTA
DELETE	/v1/historico-paciente/{id}	ADMINISTRADOR
GET	/v1/agendamento	MEDICO, RECEPCIONISTA, ENFERMEIRO, PACIENTE
POST	/v1/agendamento	MEDICO, RECEPCIONISTA, ENFERMEIRO
PATCH	/v1/agendamento/{id}	RECEPCIONISTA
DELETE	/v1/agendamento/{id}	RECEPCIONISTA

As escritas respondem `MensagemSucessoResponse` e as listagens `PageResponse<XDTO>`. A documentação interativa está em </AgendamentoAPI/swagger-ui/index.html>.

Regras de negócio

- **Atualização parcial (PATCH).** Nenhum campo é obrigatório na atualização: envie apenas o que deseja alterar. Campos ausentes, assim como strings vazias ou só com espaços, preservam o valor atual. As validações de *formato* (e-mail, tamanho de CPF/telefone) continuam valendo para os campos enviados preenchidos.
- **Modo estrito do Jackson.** Um campo desconhecido no JSON derruba a requisição com **400**.
- **Exclusão lógica.** Excluir médico, enfermeiro, recepcionista ou paciente troca a `situacaoCadastro` para **EXCLUIDO** e desativa as credenciais. A partir daí o login responde **403** e o token emitido antes é recusado com **401**. Histórico e agendamento não têm situação de cadastro, então sua exclusão é física.
- **Agenda do médico.** Ao marcar ou remarcar, exige-se um **intervalo mínimo de 1 hora** entre as consultas do médico. Um horário dentro do intervalo responde **409 Horário Indisponível!**. Exatamente 1 hora antes ou depois é aceito. Na remarcação, a própria consulta é desconsiderada.
- **Agenda do paciente.** Não é possível marcar nem remarcar se o paciente já tiver outra consulta no **mesmo** horário, e nesse caso responde **409 Horário de Consulta Indisponível!**.
- **Paginação.** Numeração baseada em 1, com os parâmetros `page`, `size` e `sort`.
- **Datas.** `LocalDate` como `dd/MM/yyyy` e `LocalDateTime` como `dd/MM/yyyy - HH:mm:ss`.

Mensageria de eventos de agendamento

Ao **criar** (POST) ou **remarcar** (PATCH) uma consulta, o serviço publica um evento na exchange **agendamento.events** (tipo *direct*), que roteia para a fila **notificacao.email.agendamento**. Um serviço consumidor, por exemplo de notificação por e-mail, pode reagir ao evento sem que a AgendamentoAPI o conheça.

Evento	Routing key	Type-Id
Consulta criada	agendamento.criado	agendamento.criado.v1
Consulta remarcada	agendamento.atualizado	agendamento.atualizado.v1

Configuração. Host, porta e credenciais do broker vêm das variáveis **RABBITMQ_HOST**, **RABBITMQ_PORT**, **RABBITMQ_USERNAME** e **RABBITMQ_PASSWORD**. A exchange, a fila e os bindings são declarados pela aplicação na primeira publicação, e as mensagens são duráveis, em JSON.

Exemplos de requisição e resposta

1) Autenticação

```
POST /AgendamentoAPI/v1/auth/login
Content-Type: application/json
```

```
{
  "login": "fernanda.lima",
  "senha": "Teste@123"
}
```

```
HTTP/1.1 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "tipo": "Bearer"
}
```

Credenciais do *seed* para teste local (todas com a senha **Teste@123**): **admin** (administrador), **joao.silva** (médico), **fernanda.lima** (repcionista), **pedro.almeida** (paciente). O token vai no header **Authorization: Bearer <token>**.

2) Agendar uma consulta

POST /AgendamentoAPI/v1/agendamento

Authorization: Bearer <token>

Content-Type: application/json

```
{
  "medicoId": 1,
  "pacienteId": 1,
  "dataHoraConsulta": "25/08/2027 - 14:00:00",
  "observacao": "Primeira consulta cardiológica."
}
```

HTTP/1.1 201 Created

```
{
  "status": 201,
  "timestamp": "12/09/2026 - 15:32:58",
  "mensagem": "Consulta criada com sucesso!"
}
```

3) Evento publicado no RabbitMQ

```
// routing key: agendamento.criado • header __TypeId__: agendamento.criado.v1
{
  "eventId": "a9ab74d4-6cd2-4f20-8b4d-5a7aeee589ea",
  "agendamentoId": 10,
  "pacienteId": 1,
  "pacienteNome": "PEDRO ALMEIDA",
  "pacienteEmail": "pedro.almeida@email.com",
  "medicoId": 1,
  "medicoNome": "JOAO SILVA",
  "especialidade": "CARDIOLOGIA",
  "dataHoraConsulta": "2027-08-25T14:00:00",
  "observacao": "Primeira consulta cardiológica.",
  "ocorridoEm": "2026-09-12T15:32:58"
}
```

Erros comuns

Código	Quando ocorre
400	Corpo inválido, campo desconhecido ou formato incorreto (validação).
401	Token ausente, inválido, expirado ou de usuário excluído.
403	Perfil sem permissão para a operação, ou usuário inativo no login.
409	Conflito de agenda (médico ou paciente indisponível no horário).

Todo erro sai no formato **ErrorResponseDTO** com **status**, **title**, **instance**, **type**, **detail** e **timestamp**, produzido pelo **GlobalExceptionHandler**.

4. HistoricoAPI: Serviço de Histórico

API GraphQL, somente leitura, que consolida em uma única consulta os dados cadastrais do paciente, o histórico clínico e as consultas agendadas. Todas as rotas são relativas ao context path **/HistoricoAPI**.

Estrutura de pastas

```
src/main/java/br/com/fiap/historicoapi/
├─ config/           # DataBaseConfig (dev, prod), SwaggerConfig
├─ controller/       # HistoricoController: resolver GraphQL
├─ service/          # HistoricoService: orquestra os repositórios
├─ repository/       # paciente, agendamento, historicopaciente
├─ model/
│   ├─ entity/       # entidades JPA (mapeiam o schema existente)
│   └─ dto/          # records de saída
├─ exceptions/       # exceções, DTOs de erro e handler global
└─ util/             # FormatadorData

src/main/resources/graphql/      # query.graphqls, schema.graphqls
src/test/resources/db/migration/ # migrations do perfil test (banco autossuficiente)
```

Endpoints

Por ser GraphQL, há um **único endpoint de dados**: a operação vai no corpo da requisição.

Endpoint	Método	Descrição
/HistoricoAPI/graphql	POST	Endpoint único de dados. Recebe query , operationName e variables , e devolve o envelope { data, errors } .
/HistoricoAPI/graphiql	GET	Interface interativa GraphiQL, para explorar o schema no navegador.
/HistoricoAPI/swagger-ui/index.html	GET	Documentação Swagger UI.
/HistoricoAPI/v3/api-docs	GET	Documento OpenAPI bruto, em JSON.

Operação GraphQL

Operação	Tipo	Argumento	Retorno
getHistoricoPaciente	Query	pacienteId: ID!	Paciente com dados cadastrais e as listas historico e consultas .

Erros previstos

Situação	Classificação	Mensagem
Id nulo, zero ou negativo	BAD_REQUEST	O ID do Paciente deve ser um número inteiro positivo!

Situação	Classificação	Mensagem
Paciente inexistente	NOT_FOUND	Paciente não encontrado - ID: {id}
Falha inesperada	INTERNAL_ERROR	Mensagem genérica. O detalhe fica apenas no log.

Por que o status é sempre 200. Seguindo a especificação GraphQL, o transporte HTTP responde **200 OK** mesmo quando a execução falha. O problema é sinalizado no array **errors**, com a classificação em **extensions.classification**. Requisições que sequer chegam ao resolver (uma URL inexistente, por exemplo) são tratadas pelo pipeline Spring MVC e respondem com o **ErrorResponseDTO** e o status HTTP correspondente.

Exemplos de requisição e resposta

Requisição: consulta do histórico

```
POST /HistoricoAPI/graphql
Content-Type: application/json

{
  "query": "query($id: ID!) { getHistoricoPaciente(pacienteId: $id) {
    \" id nome sobrenome cpf email telefone dataNascimento situacaoCadastro\"
    \" historico { queixaPrincipal historicoDoenca medicamentos alergias }\"
    \" consultas { nomeMedico dataHoraConsulta observacao } } }\",
  \"variables\": { \"id\": 1 }
}
```

Resposta 200 OK, paciente encontrado

```
{
  \"data\": {
    \"getHistoricoPaciente\": {
      \"id\": \"1\",
      \"nome\": \"PEDRO\",
      \"sobrenome\": \"ALMEIDA\",
      \"cpf\": \"12345678901\",
      \"email\": \"pedro.almeida@email.com\",
      \"situacaoCadastro\": \"ATIVO\",
      \"historico\": [
        { \"queixaPrincipal\": \"Dor no peito\",
          \"medicamentos\": \"Losartana 50mg\",
          \"alergias\": \"Nenhuma alergia conhecida.\" }
      ],
      \"consultas\": [
        { \"nomeMedico\": \"JOAO\",
          \"dataHoraConsulta\": \"22/08/2026 - 08:00:00\",
          \"observacao\": \"Primeira consulta cardiológica.\" }
      ]
    }
  }
}
```

Resposta para paciente inexistente

```
// variables: { "id": 999 }  
{  
  "errors": [  
    { "message": "Paciente não encontrado - ID: 999",  
      "path": [ "getHistoricoPaciente" ],  
      "extensions": { "classification": "NOT_FOUND" } }  
  ],  
  "data": { "getHistoricoPaciente": null }  
}
```

Formato dos dados. Campos **ID!** trafegam como String pela especificação GraphQL. As datas já saem formatadas pelo utilitário **FormatadorData**, nos padrões **dd/MM/yyyy** e **dd/MM/yyyy - HH:mm:ss**. Os dados lidos existem porque foram criados pelas migrations e pelos endpoints da AgendamentoAPI.

5. Configuração e Execução

Arquivos Docker Compose

Cada repositório traz Compose com papéis distintos. A separação é intencional: a infraestrutura não pertence a nenhum serviço em particular, então subir uma API não pode implicar subir um banco novo e isolado.

Arquivo	Papel
<code>docker-compose-postgres-dev.yml</code>	Infraestrutura de desenvolvimento, com credenciais fixas e sem <code>.env</code> . Na AgendamentoAPI inclui PostgreSQL e RabbitMQ. Na HistoricoAPI, apenas o PostgreSQL. Cria a rede <code>shared-net</code> .
<code>docker-compose-postgres-prod.yml</code>	Infraestrutura de produção: lê o <code>.env</code> e possui healthchecks. Mesma divisão: broker só na AgendamentoAPI.
<code>docker-compose-agendamentoapi.yml</code>	Apenas a AgendamentoAPI, no perfil <code>prod</code> , publicando <code>9027:9027</code> . Consome <code>shared-net</code> como rede externa.
<code>docker-compose-historicoapi.yml</code>	Apenas a HistoricoAPI, no perfil <code>prod</code> , publicando <code>9028:9027</code> . Consome <code>shared-net</code> como rede externa.

A definição do PostgreSQL é idêntica entre os dois repositórios, com o mesmo nome de projeto Compose e de volume, então o container e os dados são sempre os mesmos, independentemente de qual repositório inicia a infraestrutura. Suba-a **uma única vez**. O RabbitMQ, por ser exclusivo da AgendamentoAPI, aparece só nos Compose dela.

Configuração do ambiente (.env)

Em produção, as credenciais vêm de um arquivo `.env` na raiz de cada projeto. As variáveis criam a infraestrutura e conectam a API, de modo que não têm como divergir.

```
# AgendamentoAPI
DATABASE_NAME=postgres
DATABASE_USER=postgres
DATABASE_PASSWORD=*****
RABBITMQ_USERNAME=fiap
RABBITMQ_PASSWORD=*****
JWT_SECRET=uma-chave-longa-e-secreta
JWT_EXPIRATION_MS=3600000

# HistoricoAPI
DATABASE_NAME=postgres
DATABASE_USER=postgres
DATABASE_PASSWORD=*****
```

Segredos. As senhas e o `JWT_SECRET` vivem apenas no `.env` (fora do controle de versão), nunca no código. Em produção, defina sempre um `JWT_SECRET` próprio.

Instruções para execução local

Pré-requisitos: **Java 21** (a toolchain do Gradle resolve a versão), **Docker** e **Docker Compose**, e os dois repositórios clonados lado a lado. A `AgendamentoAPI` executa as migrations e cria o schema, então deve rodar antes da `HistoricoAPI` contra um banco novo.

Desenvolvimento (aplicações pela IDE, perfil dev na porta 9017)

```
# 1. Infraestrutura de desenvolvimento (uma vez, a partir de qualquer projeto)
docker compose -f docker-compose-postgres-dev.yml up -d --wait

# 2. AgendamentoAPI: aplica as migrations e cria o schema + carga inicial
cd AgendamentoAPI && ./gradlew bootRun --args="--spring.profiles.active=dev"

# 3. HistoricoAPI: lê o mesmo banco
cd ../HistoricoAPI && ./gradlew bootRun --args="--spring.profiles.active=dev"
```

Produção (tudo em contêiner, preencher antes o `.env`)

```
# 1. Infraestrutura: cria a rede shared-net (a partir da AgendamentoAPI, que traz o broker)
cd AgendamentoAPI
docker compose -f docker-compose-postgres-prod.yml up -d --wait

# 2. AgendamentoAPI (aplica migrations, publica em 9027)
docker compose -f docker-compose-agendamentoapi.yml up -d

# 3. HistoricoAPI (publica em 9028)
cd ../HistoricoAPI
docker compose -f docker-compose-historicoapi.yml up -d
```

```
# Testes de integração (basta a infraestrutura no ar)
./gradlew test
```

Windows. Utilize `./gradlew.bat` no lugar de `./gradlew`. O IntelliJ traz as configurações prontas em `.run/` (BootRun DEV, BootRun PROD, Clean Build e Testes de Integração), que já injetam as variáveis de ambiente.

URLs e portas configuradas

Recurso	URL	Porta
AgendamentoAPI (dev / prod)	<code>http://localhost:9017/AgendamentoAPI</code> · :9027	9017 / 9027
Swagger UI (Agendamento)	<code>/AgendamentoAPI/swagger-ui/index.html</code>	9017 / 9027
HistoricoAPI (dev / prod)	<code>http://localhost:9017/HistoricoAPI</code> · :9028	9017 / 9028

Recurso	URL	Porta
GraphiQL (Histórico)	<code>/HistoricoAPI/graphiql</code>	9017 / 9028
PostgreSQL	<code>jdbc:postgresql://localhost:8745/postgres</code>	8745 / 5432
RabbitMQ (AMQP / painel)	<code>amqp://localhost:5672</code> · <code>http://localhost:15672</code>	5672 / 15672

Em cada par de portas, a primeira é a publicada no host e a segunda é a interna do contêiner ou da rede. As APIs sempre escutam em **9027** dentro do contêiner.

Perfis da aplicação

Perfil	Características
dev	SQL formatado, Hibernate em DEBUG. Log apenas no console. Porta 9017.
prod	Log em arquivo rotativo diário em <code>/app/logs</code> , com expurgo após 30 dias. Porta 9027 no contêiner.
test	Perfil da suíte de integração, roda contra banco real. Na HistoricoAPI, o único em que o Flyway está habilitado.

6. Qualidade do Código

Os dois projetos adotam as convenções idiomáticas do Spring Boot e princípios consolidados de projeto orientado a objetos. As suítes de integração servem de rede de segurança e são verificadas a cada Pull Request.

■ AgendamentoAPI

157 métodos de teste em 24 classes, com **100% de cobertura** de instruções e de branches em controllers, services e repositories.

JaCoCo · JUnit 5

■ HistoricoAPI

32 métodos de teste em 10 classes, com **100% de cobertura** de instruções, linhas, métodos, classes e branches.

JaCoCo · JUnit 5

Princípios SOLID e DRY

- **Responsabilidade única.** Cada camada tem um único motivo para mudar: o controller expõe a operação, o serviço concentra a regra de negócio, os repositórios cuidam do acesso a dados e os DTOs definem o contrato de saída.
- **Inversão de dependência.** Os serviços dependem de interfaces **JpaRepository**, nunca de implementações concretas, e quem as fornece é o container do Spring, por injeção via construtor com **@RequiredArgsConstructor** e campos **final**.
- **Aberto/fechado.** Novos domínios ou consultas entram como novas fatias verticais ou novos resolvers, sem alterar o que já existe. Uma nova exceção de negócio entra como um novo handler.
- **DRY.** O **GlobalExceptionHandler** concentra o tratamento de erros, os DTOs expõem uma *factory* única de conversão, a formatação de datas é centralizada e as classes-base de teste concentram as anotações comuns.

Modelagem, segurança e integração contínua

- **Imutabilidade e camadas isoladas.** DTOs e requests são **record** imutáveis, e a entidade JPA nunca é serializada diretamente, isolando o contrato público do mapeamento de banco.
- **Carregamento explícito.** Associações são **LAZY** e carregadas dentro da transação, evitando N+1 e **LazyInitializationException**.
- **Segurança da informação.** Senhas com BCrypt e nunca expostas, mensagens de exceções não-domínio não voltam ao cliente, e **include-stacktrace: never** impede vazamento de stacktrace. Nenhum segredo no código.
- **Testes de integração reais** sobem o contexto do Spring contra um PostgreSQL de verdade, não um banco em memória.
- **Integração contínua.** A cada Pull Request para **main**, o GitHub Actions provisiona um PostgreSQL 18, configura o Java 21 e executa **./gradlew build**, e uma regressão reprova o PR antes do merge.

7. Repositórios do Código

URLs

- **AgendamentoAPI:** <https://github.com/JavaPosTech/AgendamentoAPI>
- **HistoricoAPI:** <https://github.com/JavaPosTech/HistoricoAPI>
- Branch de entrega: **main** · Versão: **1.0.0**

Organização de cada repositório

Caminho	Conteúdo
<code>/src/main/java</code>	Código-fonte, organizado por camada e por domínio
<code>/src/main/resources</code>	<code>application.yaml</code> , <code>log4j2.xml</code> . Migrations (Agendamento) ou schemas GraphQL (Histórico)
<code>/src/test</code>	Suíte de integração e migrations do perfil de teste na HistoricoAPI
<code>docker-compose-postgres-*.yaml</code>	Infraestrutura de dev e de prod (PostgreSQL, e RabbitMQ na Agendamento)
<code>docker-compose-<api>.yaml</code>	Orquestração da própria API no perfil prod
<code>Dockerfile</code>	Build multi-stage da imagem da aplicação
<code>.github/workflows</code>	Pipeline de integração contínua a cada Pull Request
<code>README.md</code> · <code>CLAUDE.md</code>	Documentação de execução e de arquitetura do projeto

Serviços da Fase 3

Serviço	Porta (host)	Papel
AgendamentoAPI	9027	Cadastros, autenticação, agendamento e histórico. Dona das migrations e da mensageria
HistoricoAPI	9028	Consulta consolidada do histórico do paciente, por GraphQL. Somente leitura

Infraestrutura compartilhada: PostgreSQL 18 (host **8745**) e RabbitMQ (host **5672**, painel **15672**), ambos na rede Docker **shared-net**. A AgendamentoAPI é o serviço dono do schema e deve subir primeiro contra um banco novo.